

Tyr-Crypto

Computational Handbook

Volume 2: Poly1305 and ChaCha20-Poly1305

A guided calculation book for one-time keys, clamping, block accumulation, reduction modulo $2^{130} - 5$, tags, verification, and the AEAD message layout.

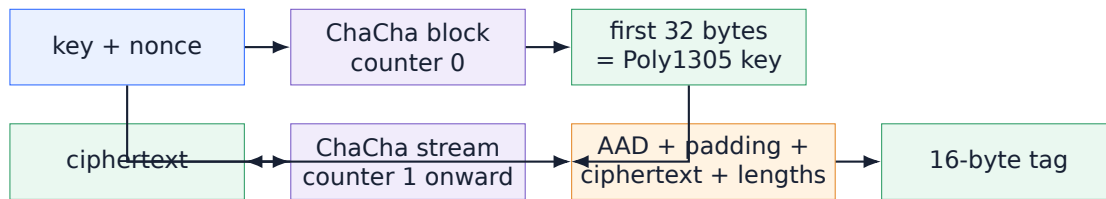
This volume distinguishes the standard RFC AEAD composition from Tyr-Crypto's current separately exposed XChaCha20 and Poly1305 primitives.

Blue	new idea, input, or plain-language explanation
Purple	exact recipe to follow
Green	fully worked calculation
Orange	exercise for the reader
Red	misuse, security, or implementation danger
Teal	checkpoint before continuing

Contents

1 The entire construction before the details	2
1.1 What Poly1305 promises	2
2 Method 1: read a byte block as a little-endian integer	3
3 Method 2: append the Poly1305 block marker	4
4 Method 3: clamp the first half of the key	5
5 Method 4: update the Poly1305 accumulator	6
6 Method 5: reduce efficiently modulo $2^n - 5$	7
7 Method 6: split a 130-bit number into five 26-bit limbs	8
8 Method 7: multiply limb vectors and propagate carries	9
9 Method 8: create the final 16-byte tag	10
10 Method 9: compare tags without early exit	11
11 Method 10: derive Poly1305's one-time key	12
12 Method 11: build the exact AEAD data authenticated by Poly1305	13
13 Method 12: authenticated encryption and safe opening	14
14 Reading the Tyr-Crypto implementation	15
15 Cumulative exercises	16
16 Solutions	17
17 References and code map	18

1 The entire construction before the details



Do not skip this warning

The current Tyr-Crypto source has `xchacha20Xor` and `poly1305Mac` as separate primitives. Its typed `XChaCha encrypt/decrypt` tests only round-trip stream encryption. A ciphertext from that wrapper is not automatically authenticated merely because the README labels the example AEAD.

1.1 What Poly1305 promises

Poly1305 accepts a 32-byte one-time key and any message, then produces a 16-byte tag. The key is split into two 16-byte numbers called r and s . The message is processed in 16-byte blocks by a modular accumulator.

$$\text{Acc} \leftarrow ((\text{Acc} + \text{block}) \cdot r) \bmod (2^{130} - 5), \quad \text{tag} = (\text{Acc} + s) \bmod 2^{128}.$$

2 Method 1: read a byte block as a little-endian integer

Input: 1 to 16 bytes
Do: attach powers 2^{8i} from left to right
Output: one nonnegative integer

Mechanical rule

For bytes $b_0, b_1, \dots, b_{t-1}$:

$$n = \sum_{i=0}^{t-1} b_i 2^{8i}.$$

The first byte is the least significant byte.

Worked example 1 - one byte

2A represents 42.

Worked example 2 - two bytes

01 02 represents $1 + 2 \cdot 256 = \boxed{513} = 0x0201$.

Worked example 3 - four bytes

78 56 34 12 represents $\boxed{0x12345678}$.

Worked example 4 - sixteen bytes

The block 00 01 02 ... 0F is written as the hexadecimal integer 0x0F0E0D0C0B0A09080706050403020100.

Exercises

Convert (a) FF 00, (b) 10 00 00 01, (c) AA BB CC DD, and (d) 01 00 00 00 00 00 00 80.

3 Method 2: append the Poly1305 block marker

Input: a message block of at most 16 bytes
Do: place one binary 1 immediately above the block
Output: an integer between 1 and 2^{129}

Plain-language explanation

Poly1305 must distinguish a short block from the same bytes followed by zeros. It therefore appends a 1 beyond the actual message bytes. Mathematically, add 2^{8t} to a block containing t bytes.

Mechanical rule

$\text{blockValue} = \text{littleEndian}(b_0 \dots b_{t-1}) + 2^{8t}$.

Worked example 1 - empty conceptual block

For $t = 0$, the marker alone is $2^0 = 1$. Real processing simply has no block when the message is empty.

Worked example 2 - one zero byte

00 gives $0 + 2^8 = \boxed{256}$. This differs from an empty block's marker 1.

Worked example 3 - bytes 01 02

The raw integer is 513. Add $2^{16} = 65536$: block value = $\boxed{66049}$.

Worked example 4 - full 16-byte block

For a full block, add 2^{128} . Tyr-Crypto represents this extra bit through the hibit in the highest 26-bit limb rather than allocating a 17th byte.

Exercises

Append the marker to (a) 01, (b) FF, (c) 00 00, and (d) a 16-byte all-zero block.

4 Method 3: clamp the first half of the key

Input: 16 bytes called raw r
Do: clear specific high and low bits
Output: the restricted multiplier r

Plain-language explanation

Clamping means forcing selected key bits to zero. It is not rounding and it is not hashing. It is a fixed bit mask.

Mechanical rule

Clear the top four bits of bytes 3,7,11,15 by ANDing with 0x0F. Clear the bottom two bits of bytes 4,8,12 by ANDing with 0xFC.

Worked example 1

Raw r : FF FF FF FF FF FF FF FF FF FF FF FF FF FF FF
 Clamped: FF FF FF 0F FC FF FF 0F FC FF FF 0F FC FF FF 0F

Worked example 2

Raw r : 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F
 Clamped: 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F

Worked example 3

Raw r : 85 D6 BE 78 57 55 6D 33 7F 44 52 FE 42 D5 06 A8
 Clamped: 85 D6 BE 08 54 55 6D 03 7C 44 52 0E 40 D5 06 08

Worked example 4

Raw r : 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F
 Clamped: 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F

Exercises

Clamp (a) sixteen 0xAA bytes, (b) sixteen 0x55 bytes, (c) 01 23 45 67 89 AB CD EF 10 32 54 76 98 BA DC FE, and (d) explain which bytes remain unchanged.

5 Method 4: update the Poly1305 accumulator

Input: old accumulator, one marked block, multiplier r
Do: add, multiply, reduce modulo p
Output: new accumulator

Plain-language explanation

The real modulus is enormous, so the first examples use the same shape with the toy modulus $p = 2^8 - 5 = 251$. Only the size changes in real Poly1305.

Mechanical rule

$\text{Acc}_{\text{new}} = ((\text{Acc}_{\text{old}} + m)r) \bmod p$. Always do the operations in this order: add block, multiply by r , reduce.

Worked example 1 - toy modulus 251

$$((0 + 7) \cdot 3) \bmod 251 = 21 \bmod 251 = \boxed{21}.$$

Worked example 2 - toy modulus 251

$$((5 + 12) \cdot 4) \bmod 251 = 68 \bmod 251 = \boxed{68}.$$

Worked example 3 - toy modulus 251

$$((250 + 10) \cdot 2) \bmod 251 = 520 \bmod 251 = \boxed{18}.$$

Worked example 4 - toy modulus 251

$$((100 + 200) \cdot 17) \bmod 251 = 5100 \bmod 251 = \boxed{80}.$$

Exercises

With $p = 251$, compute updates for (a) $(\text{Acc}, m, r) = (0, 20, 5)$, (b) $(10, 15, 7)$, (c) $(240, 30, 3)$, (d) $(100, 100, 100)$.

6 Method 5: reduce efficiently modulo $2^n - 5$

Input: a number larger than the modulus
Do: fold high powers back using $2^n \equiv 5$
Output: a smaller congruent number

Plain-language explanation

Poly1305's modulus has the special form $2^{130} - 5$. Therefore one copy of 2^{130} can be replaced by 5 during reduction. Tyr-Crypto's five 26-bit limbs use exactly this idea when a carry leaves the top limb: it returns to the bottom multiplied by 5.

Mechanical rule

Write $x = x_{low} + 2^n x_{high}$. Then

$$x \bmod (2^n - 5) \equiv x_{low} + 5x_{high}.$$

Repeat if the result is still too large.

Worked example 1 - one excess power

Modulo $2^8 - 5 = 251$, $2^8 \equiv 5$. Therefore $256 \bmod 251 = \boxed{5}$.

Worked example 2 - high part 3

$3 \cdot 256 + 20 \equiv 3 \cdot 5 + 20 = \boxed{35} \pmod{251}$.

Worked example 3 - fold twice

$1000 = 3 \cdot 256 + 232 \equiv 15 + 232 = 247$, already below 251.

Worked example 4 - real limb meaning

A carry c beyond bit 129 represents $c \cdot 2^{130}$. Replace it by $5c$ added to limb 0, then propagate any new carry.

Exercises

Reduce modulo 251 using folding: (a) 300, (b) 512, (c) 1024, (d) $7 \cdot 256 + 250$.

7 Method 6: split a 130-bit number into five 26-bit limbs

Input: one integer h
Do: take groups of 26 bits from low to high
Output: five smaller integers $h_0..h_4$

Plain-language explanation

A limb is simply one chunk of a larger integer. Five chunks of 26 bits provide 130 bits. Smaller chunks let 64-bit variables hold products plus carries without immediately overflowing.

Mechanical rule

$h = h_0 + 2^{26}h_1 + 2^{52}h_2 + 2^{78}h_3 + 2^{104}h_4$, with each $0 \leq h_i < 2^{26}$.

Worked example 1

$h = 0$ gives $(h_0, h_1, h_2, h_3, h_4) = (0, 0, 0, 0, 0)$.

Worked example 2

$h = 67108863$ gives $(h_0, h_1, h_2, h_3, h_4) = (67108863, 0, 0, 0, 0)$.

Worked example 3

$h = 67108867$ gives $(h_0, h_1, h_2, h_3, h_4) = (3, 1, 0, 0, 0)$.

Worked example 4

$h = 20282409603651670423947251286023$ gives $(h_0, h_1, h_2, h_3, h_4) = (7, 0, 0, 0, 1)$.

Exercises

Split: (a) $2^{26} + 1$, (b) $2^{52} + 2^{26} + 1$, (c) $2^{104} - 1$, (d) $3 \cdot 2^{104} + 2$.

8 Method 7: multiply limb vectors and propagate carries

Input: five accumulator limbs and five r limbs
Do: fixed cross-products, then base- 2^{26} carries
Output: five reduced limbs

Plain-language explanation

The source precomputes $s_1 = 5r_1, \dots, s_4 = 5r_4$. Terms that would cross the 2^{130} boundary use these multiplied-by-5 values immediately. This fuses multiplication with the special-modulus reduction.

Mechanical rule

The first temporary is

$$d_0 = h_0 r_0 + h_1(5r_4) + h_2(5r_3) + h_3(5r_2) + h_4(5r_1).$$

Then $h_0 = d_0 \bmod 2^{26}$ and carry $c = \lfloor d_0/2^{26} \rfloor$ is added to d_1 . Repeat through d_4 , then fold the top carry into h_0 as $5c$.

Worked example 1 - base-16 toy carry

Using tiny 4-bit limbs, $d_0 = 35$ gives limb $35 \bmod 16 = 3$ and carry $\lfloor 35/16 \rfloor = 2$.

Worked example 2 - carry chain

Toy temporaries (35, 14): first becomes limb 3 and adds carry 2 to the next, making 16; next becomes limb 0 with carry 1.

Worked example 3 - top fold

If the top toy limb emits carry 3 under modulus $2^{20} - 5$, add $5 \cdot 3 = 15$ to limb 0.

Worked example 4 - real mask

Tyr-Crypto uses mask $0x03FFFFFF$, which equals $2^{26} - 1$, to keep the low 26 bits after each carry extraction.

Exercises

With 4-bit toy limbs, propagate carries through (a) (20, 0), (b) (31, 31), (c) (15, 16, 17), and (d) explain why a fixed carry schedule is useful for timing behavior.

9 Method 8: create the final 16-byte tag

Input: final accumulator and second key half s
Do: canonical reduction, add s , keep 128 low bits, serialize
Output: 16 tag bytes

Mechanical rule

$$T = (h + s) \bmod 2^{128}.$$

The addition uses four 32-bit words with carry. The output words are stored little-endian.

Worked example 1 - no carry

$h = 10$, $s = 20$ gives tag integer 30.

Worked example 2 - 8-bit toy truncation

With an 8-bit tag, $h = 250$, $s = 20$ gives $270 \bmod 256 = \boxed{14}$.

Worked example 3 - carry between 32-bit words

Low word $0xFFFFFFFF + 0x00000001$ becomes $0x00000000$ and sends carry 1 into the next word.

Worked example 4 - official tag

RFC 8439's message "Cryptographic Forum Research Group" and one-time key produce tag A8 06 1D C1 30 51 36 C6 C2 2B 8B AF 0C 01 27 A9.

Exercises

(a) Add two 8-bit toy tag values 240 and 30. (b) Serialize low word $0xC11D06A8$. (c) Explain why only 128 bits are output although the modulus is near 130 bits. (d) What is the tag length in Tyr-Crypto?

10 Method 9: compare tags without early exit

Input: expected tag and received tag
Do: XOR all positions and OR the differences
Output: equal or unequal

Plain-language explanation

A normal comparison may stop at the first differing byte. Its runtime can then reveal the location of the first match failure. A constant-time-shaped comparison processes the whole public tag length.

Mechanical rule

Start $diff = 0$. For every i , set $diff \leftarrow diff \text{ OR } (A_i \oplus B_i)$. Equal only when final $diff = 0$.

Worked example 1 - equal one-byte tags

$0xA5 \text{ XOR } 0xA5 = 0$; final diff 0 means equal.

Worked example 2 - first byte differs

00 11 22 versus 01 11 22: process all three bytes even though the first differs.

Worked example 3 - last byte differs

AA BB CC versus AA BB CD: earlier XORs are zero; final XOR is 1, so diff is nonzero.

Worked example 4 - length differs

Tyr-Crypto seeds diff with 1 when lengths differ, then still scans the expected tag bytes using zero for missing received bytes.

Exercises

Trace diff for (a) 01 02 vs 01 02, (b) 01 02 vs 00 02, (c) FF 00 vs FF 01, (d) explain why returning immediately is undesirable.

11 Method 10: derive Poly1305's one-time key

Input:	session key and nonce
Do:	generate ChaCha block 0 and take its first 32 bytes
Output:	$r s$ for one message

Mechanical rule

Block counter 0 is reserved for the Poly1305 one-time key. Plaintext encryption begins with counter 1. Never use the block-0 bytes both as MAC key material and plaintext keystream.

Worked example 1 - byte allocation

ChaCha block 0 has 64 bytes. Bytes 0–15 are raw r , bytes 16–31 are s , and bytes 32–63 are discarded for this construction.

Worked example 2 - counter separation

A 10-byte plaintext uses the first 10 keystream bytes of block 1, not block 0.

Worked example 3 - second message

A new nonce creates a new block-0 output and therefore a new Poly1305 key even when the session key stays the same.

Worked example 4 - XChaCha version

For XChaCha20-Poly1305, first derive the HChaCha subkey and 12-byte nonce as in Volume 1. Then apply exactly the same counter-0/counter-1 allocation under that subkey.

Exercises

(a) Which 32 bytes form the one-time key? (b) Which block encrypts plaintext byte 0? (c) Why must nonce reuse be prevented? (d) What happens to block-0 bytes 32–63?

12 Method 11: build the exact AEAD data authenticated by Poly1305

Input: AAD and ciphertext
Do: concatenate with 16-byte padding and two 64-bit lengths
Output: one MAC input byte string

Mechanical rule

$$\text{macData} = \text{AAD} \parallel \text{pad16}(\text{AAD}) \parallel C \parallel \text{pad16}(C) \parallel \text{len64}(\text{AAD}) \parallel \text{len64}(C).$$

Lengths are byte counts encoded as 8-byte little-endian integers.

Worked example 1 - no AAD, no ciphertext

Both data strings have length 0 and need no padding. The MAC input is sixteen zero bytes: two 8-byte length fields.

Worked example 2 - 3 AAD bytes

AAD AA BB CC receives 13 zero padding bytes. If ciphertext is empty, append no ciphertext padding, then length fields 03 00...00 and 00 00...00.

Worked example 3 - exactly 16 bytes

A 16-byte AAD needs zero padding bytes, not another full padding block.

Worked example 4 - AAD 5, ciphertext 18

AAD gets 11 zeros; ciphertext gets 14 zeros. Then append little-endian 64-bit values 5 and 18.

Exercises

Find padding counts for lengths (a) 0, (b) 1, (c) 15, (d) 16, (e) 17. Then draw the complete MAC layout for AAD length 2 and ciphertext length 3.

13 Method 12: authenticated encryption and safe opening

Input:	key, nonce, plaintext, AAD
Do:	derive key, encrypt, tag; or verify then release plaintext
Output:	ciphertext+tag or authenticated plaintext

Worked example 1 - encryption order

Derive one-time key with counter 0; encrypt with counter 1; build MAC data from AAD and ciphertext; compute tag; return ciphertext and tag.

Worked example 2 - successful opening

Rebuild the one-time key and MAC data, compute expected tag, compare all bytes, then decrypt and release plaintext.

Worked example 3 - modified ciphertext

One flipped ciphertext bit changes the MAC input. Tag comparison should fail; the application must receive no unauthenticated plaintext.

Worked example 4 - modified AAD

AAD is not encrypted, but it is authenticated. Changing a header byte causes tag failure even when ciphertext is unchanged.

Do not skip this warning

Some low-level APIs decrypt before verifying for implementation reasons, but unauthenticated plaintext must remain quarantined and must never influence parsing, errors, or application behavior. The safest teaching rule is: verify before releasing plaintext.

Exercises

For each change, say whether verification must fail: (a) ciphertext bit flip, (b) AAD bit flip, (c) tag bit flip, (d) nonce change. Then state what may be released to the caller on failure.

14 Reading the Tyr-Crypto implementation

Source calculation	Book method
load32Le and overlapping loads	Methods 1 and 6: little-endian chunks and 26-bit limb extraction.
key masks at r0..r4	Method 3: clamping, expressed directly in limb form.
h0..h4 and d0..d4	Methods 6 and 7: accumulator limbs and cross-products.
carry sequence with shr 26	Methods 5 and 7: base- 2^{26} propagation and fold by 5.
finalizeTag	Method 8: canonical reduction, add s , serialize four 32-bit words.
constantTimeEqual	Method 9: XOR/OR comparison without early exit.

Do not skip this warning

The current source does not contain a combined aeadXChaCha20Poly1305 operation. The standard composition in Methods 10–12 is therefore a design reference, not a claim that the current typed cipher wrapper already performs it.

15 Cumulative exercises

Cumulative A - toy Poly1305 over two blocks

Use toy modulus 251, $r = 7$, $s = 20$, and marked block values $m_1 = 30$, $m_2 = 80$. Start at accumulator 0, process both blocks, then form an 8-bit toy tag by adding s modulo 256.

Cumulative B - build an AEAD MAC input

AAD is 01 02 03 04 05; ciphertext is A0 A1 A2. Write the full layout, padding counts, and two 8-byte length fields.

Cumulative C - diagnose a wrapper

A function calls only `xchacha20Xor` and returns its output. List exactly which AEAD steps are absent and what attacks remain possible.

Cumulative D - key lifetime

Draw the lifetimes of session key, HChaCha subkey, Poly1305 one-time key, accumulator limbs, plaintext, ciphertext, and tag. Mark which values should be wiped earliest.

16 Solutions

Methods 1-3: block integers, marker, and clamping

Method 1. Attach powers 2^{8i} from left to right: (a) $0x00FF$; (b) $0x01000010$; (c) $0xDDCCBBAA$; (d) $0x8000000000000001$.

Method 2. The marker is one bit immediately above the received bytes: (a) $1 + 2^8 = 257$; (b) $255 + 2^8 = 511$; (c) 2^{16} ; (d) a full 16-byte zero block represents 2^{128} .

Method 3. Apply $0x0F$ to bytes 3,7,11,15 and $0xFC$ to bytes 4,8,12:

(a) AA AA AA 0A A8 AA AA 0A A8 AA AA 0A A8 AA AA 0A;

(b) 55 55 55 05 54 55 55 05 54 55 55 05 54 55 55 05;

(c) 01 23 45 07 88 AB CD 0F 10 32 54 06 98 BA DC 0E.

(d) Every byte except positions 3,4,7,8,11,12,15 remains byte-for-byte unchanged.

Methods 4-7: accumulator arithmetic and limbs

Method 4. Use $((Acc + m)r) \bmod 251$: (a) 100; (b) 175; (c) $270 \cdot 3 = 810 \equiv 57$; (d) $200 \cdot 100 = 20000 \equiv 171$.

Method 5. Replace each 256 by 5: (a) $300 = 256 + 44 \mapsto 49$; (b) $512 = 2 \cdot 256 \mapsto 10$; (c) $1024 = 4 \cdot 256 \mapsto 20$; (d) $7 \cdot 256 + 250 \mapsto 35 + 250 = 285 \equiv 34$.

Method 6. (a) $(1, 1, 0, 0, 0)$; (b) $(1, 1, 1, 0, 0)$; (c) $(2^{26} - 1, 2^{26} - 1, 2^{26} - 1, 2^{26} - 1, 0)$; (d) $(2, 0, 0, 0, 3)$.

Method 7. Use base 16 for the toy limbs. (a) $(20, 0) \rightarrow (4, 1)$. (b) First pass gives $(15, 0)$ and top carry 2; fold $5 \cdot 2 = 10$ into limb 0, giving $(9, 1)$. (c) $(15, 16, 17)$ gives $(15, 0, 2)$ and top carry 1; folding 5 yields $(4, 1, 2)$. (d) A fixed carry schedule executes the same number and order of operations for every secret value, avoiding a direct timing signal from carry size.

Methods 8-12: tags and authenticated composition

Method 8. (a) $(240 + 30) \bmod 256 = 14$. (b) $0xC11D06A8$ serializes as A8 06 1D C1. (c) The standardized tag is the low 128 bits after adding s ; the extra accumulator bits belong to modular arithmetic, not tag length. (d) 16 bytes.

Method 9. (a) All XORs are zero, so $\text{diff}=0$. (b) The first XOR is 1, so final diff is nonzero. (c) The last XOR is 1, also nonzero. (d) Returning at the first mismatch makes runtime depend on the matching prefix and can reveal information about the expected tag.

Method 10. (a) Bytes 0-31 of ChaCha block 0 form the one-time key. (b) Plaintext byte 0 uses block counter 1. (c) Reusing key and nonce repeats both this MAC key and the encryption stream. (d) Block-0 bytes 32-63 are not used by the standard AEAD construction.

Method 11. Padding counts for lengths 0,1,15,16,17 are 0,15,1,0,15. For AAD A0 A1 and ciphertext C0 C1 C2, the MAC input is: 2 AAD bytes, 14 zeros, 3 ciphertext bytes, 13 zeros, 02 00 00 00 00 00 00 00, then 03 00 00 00 00 00 00 00.

Method 12. Ciphertext, AAD, tag, or nonce changes all make verification fail. On failure release no plaintext and no mismatch position; return only a generic authentication failure.

Cumulative solutions

A. $Acc_1 = (30 \cdot 7) \bmod 251 = 210$. Then $Acc_2 = ((210 + 80) \cdot 7) \bmod 251 = 22$. Toy tag = $(22 + 20) \bmod 256 = 42$.

B. AAD: 01 02 03 04 05 plus 11 zeros. Ciphertext: A0 A1 A2 plus 13 zeros. Lengths: 05 00 00 00 00 00 00 00 and 03 00 00 00 00 00 00 00.

C. A bare `xchacha20Xor` call omits one-time-key derivation for Poly1305, counter separation, AAD binding, the padded length layout, tag creation, constant-time verification, and plaintext quarantine. Attackers can flip chosen plaintext bits predictably; nonce reuse also exposes plaintext relations.

D. The session key may live for the session. HChaCha subkey and Poly1305 key live for one invocation. Accumulator limbs live only during MAC computation. During opening, tentative plaintext remains private until tag success. Wipe secret temporaries immediately after their final use; ciphertext and tag are public outputs.

17 References and code map

Source	Link
Tyr Poly1305 source	https://github.com/siriuslee69/Tyr-Crypto/blob/fb9cf27926140e95c02362d4felc349ff65d3dd0/src/protocols/custom_crypto/symmetric/poly1305/poly1305.nim
Tyr XChaCha20 source	https://github.com/siriuslee69/Tyr-Crypto/blob/fb9cf27926140e95c02362d4felc349ff65d3dd0/src/protocols/custom_crypto/symmetric/chacha/xchacha20.nim
Tyr typed wrapper test	https://github.com/siriuslee69/Tyr-Crypto/blob/fb9cf27926140e95c02362d4felc349ff65d3dd0/tests/test_quick_api.nim
RFC 8439	https://www.rfc-editor.org/rfc/rfc8439
XChaCha draft	https://datatracker.ietf.org/doc/html/draft-irtf-cfrg-xchacha-03
Original Poly1305 paper	https://cr.yp.to/mac/poly1305-20050329.pdf

Do not skip this warning

This handbook explains calculations and repository structure. It is not a security certification. Do not deploy custom cryptographic code solely because worked examples match.